

LAPORAN PRAKTIKUM 5: ABSTRACT CLASS & INTERFACE



DISUSUN OLEH:

Muhammad Rif'q Apriliansyah (251511023)

2A - D3

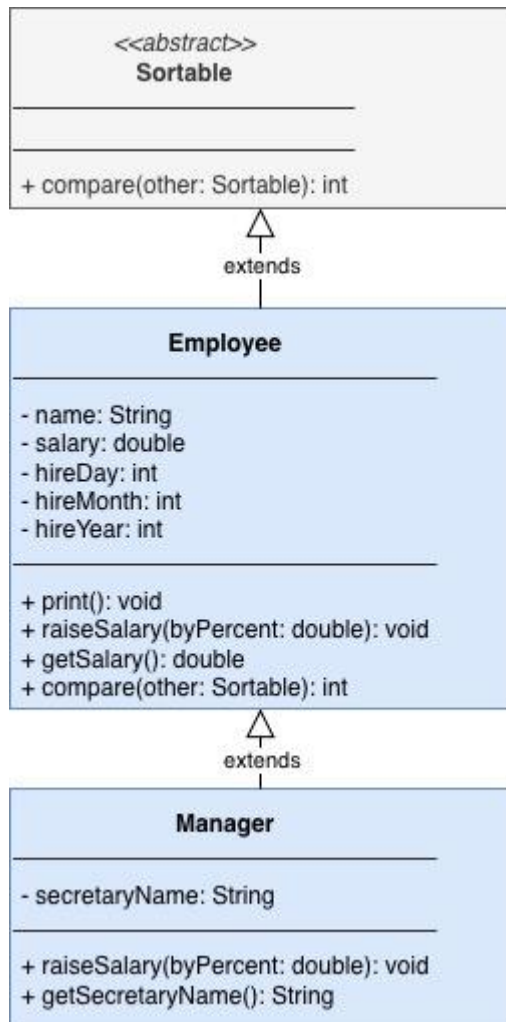
POLITENIK NEGERI BANDUNG

TEKNIK KOMPUTER & INFORMATIKA / D3 TEKNIK INFORMATIKA

2026

1. BAGIAN A - EMPLOYEE, MANAGER, SORTABLE

1.1 Diagram Hierarchy



1.2 Code Inti

A. Sortable.java

```
public abstract class Sortable {  
  
    public abstract int compare(Sortable other);  
  
}
```

B. Employee.java

```
public class Employee extends Sortable {
```

```

private String name;

private double salary;

private int hireDay, hireMonth, hireYear;

public Employee(String name, double salary, int day, int month, int year) {

    this.name = name;

    this.salary = salary;

    this.hireDay = day;

    this.hireMonth = month;

    this.hireYear = year;

}

public void print() {

    System.out.println(name + " " + salary + " " + hireDay + "/" + hireMonth + "/" + hireYear());

}

public void raiseSalary(double byPercent) {

    salary *= 1 + byPercent / 100;

}

public int hireYear() { return hireYear; }

public double getSalary() { return salary; }

@Override

public int compare(Sortable other) {

    Employee otherEmployee = (Employee) other;

    if (getSalary() < otherEmployee.getSalary()) return -1;

    if (getSalary() > otherEmployee.getSalary()) return 1;

    return 0;

```

```
}  
  
}
```

C. Manager . java

```
import java.util.Calendar;  
  
import java.util.GregorianCalendar;  
  
public class Manager extends Employee {  
  
    private String secretaryName;  
  
    public Manager(String name, double salary, int day, int month, int year) {  
  
        super(name, salary, day, month, year);  
  
        secretaryName = "";  
  
    }  
  
    @Override  
  
    public void raiseSalary(double byPercent) {  
  
        GregorianCalendar today = new GregorianCalendar();  
  
        int currentYear = today.get(Calendar.YEAR);  
  
        double bonus = 0.5 * (currentYear - hireYear());  
  
        super.raiseSalary(byPercent + bonus);  
  
    }  
  
    public String getSecretaryName() { return secretaryName; }  
  
}
```

1.3 Output EmployeeTest

Antonio Rossi 2100000.0 1/10/1989

Maria Bianchi 3062500.0 1/12/1991

Isabel Vidal 3150000.0 1/11/1993

-1

-1

1

-1

Caption: Output program EmployeeTest yang menampilkan data karyawan setelah kenaikan gaji dan hasil perbandingan gaji antar staf.

1.4 Bukti Eksperimen Abstract Class Tidak Dapat Diinstansiasi

// Eksperimen pada main():

```
Sortable s = new Sortable(); // ERROR!
```

Pesan Error Compiler:

```
error: Sortable is abstract; cannot be instantiated
```

Caption: Pengujian instansiasi langsung pada abstract class Sortable menghasilkan compile-time error karena abstract class tidak dapat dibuat objeknya.

1.5 Bukti Employee Wajib Mengimplementasikan compare ()

Jika klausa compare () dihapus dari Employee:

Pesan Error Compiler:

```
error: Employee is not abstract and does not override abstract method  
compare(Sortable) in Sortable
```

Caption: Class konkrit yang mewarisi abstract class wajib meng-override seluruh abstract method yang dimiliki oleh superclass-nya.

1.6 Hasil Perbandingan compare ()

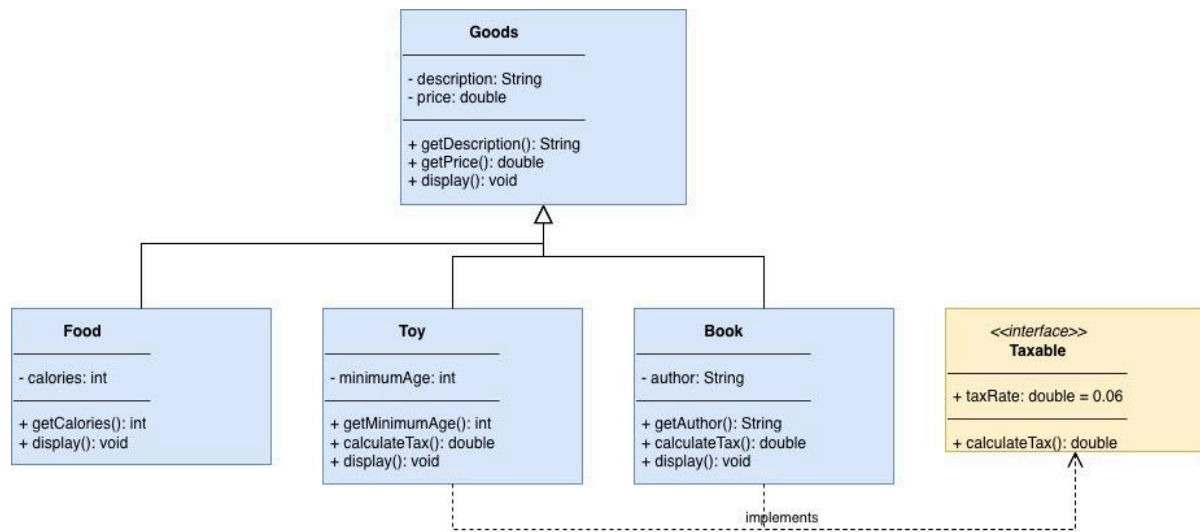
- `staff[0].compare(staff[1])`: Output -1 (Gaji Antonio Rossi < Maria Bianchi)
- `staff[1].compare(staff[2])`: Output -1 (Gaji Maria Bianchi < Isabel Vidal)
- `staff[2].compare(staff[0])`: Output 1 (Gaji Isabel Vidal > Antonio Rossi)
- `manager.compare(employee)`: Output -1 (Gaji Manager < Isabel Vidal)

1.7 Bukti Inheritance Chain Manager

Manager mewarisi Employee, dan Employee mewarisi Sortable. Oleh karena itu, secara otomatis Manager adalah subtype dari Sortable tanpa perlu menuliskan ganda `extends Employee, Sortable`.

2. BAGIAN B - GOODS DAN TAXABLE

2.1 Diagram Hierarchy + Interface



2.2 Code Inti

A. Goods . java

```
public class Goods {

    private String description;

    private double price;

    public Goods(String description, double price) {

        this.description = description;

        this.price = price;

    }

    public String getDescription() { return description; }

    public double getPrice() { return price; }

    public void display() {

        System.out.println("Description : " + description);

        System.out.println("Price      : " + price);

    }

}
```

```
}  
  
}
```

B. Taxable.java

```
public interface Taxable {  
  
    double taxRate = 0.06;  
  
    double calculateTax();  
  
}
```

C. Food.java, Toy.java, Book.java

```
a  
public class Food extends Goods {  
    private int calories; public Food(String description, double price, int calories) {  
  
        super(description, price);  
  
        this.calories = calories;  
  
    }  
  
    @Override
```

```
public void display() {  
  
    super.display();  
  
    System.out.println("Calories   : " + calories);  
  
}
```

```
}  
  
public class Toy extends Goods implements Taxable {  
    private int minimumAge; public Toy(String description, double price, int minimumAge) {  
  
        super(description, price);  
  
        this.minimumAge = minimumAge;  
  
    }  
  
    @Override
```

```
public double calculateTax() { return getPrice() * taxRate; }
```

```
@Override
```

```
public void display() {
```

```
    super.display();
```

```
    System.out.println("Minimum age : " + minimumAge);
```

```
    System.out.println("Tax      : " + calculateTax());
```

```
}
```

```
}
```

```
public class Book extends Goods implements Taxable {
```

```
    private String author; public Book(String description, double price, String author) {
```

```
        super(description, price);
```

```
        this.author = author;
```

```
}
```

```
@Override
```

```
public double calculateTax() { return getPrice() * taxRate; }
```

```
@Override
```

```
public void display() {
```

```
    super.display();
```

```
    System.out.println("Author    : " + author);
```

```
    System.out.println("Tax      : " + calculateTax())
```

```
;
```

```
}
```

```
}
```

```
### 2.3 Output `GoodsTest`
```

```
```text
```

```
Description : Bread
```

Price : 15000.0

Calories : 250

Description : Robot

Price : 120000.0

Minimum age : 8

Tax : 7200.0

Description : Java Basics

Price : 90000.0

Author : A. Programmer

Tax : 5400.0

Toy tax : 7200.0

Book tax : 5400.0

*Caption: Hasil eksekusi GoodsTest menampilkan deskripsi produk beserta perhitungan pajak sebesar 6% untuk Toy dan Book.*

## 2.4 Bukti Contract Interface

Jika method `calculateTax()` pada kelas Book atau Toy dihapus:

### Pesan Error Compiler:

```
error: Book is not abstract and does not override abstract method
calculateTax() in Taxable
```

*Caption: Kegagalan mengimplementasikan method interface menyebabkan error kompilasi karena kontrak belum dipenuhi.*

## 3. TABEL SKENARIO UJI (T5-01 s.d. T5-08)

ID Skenario	Skenario Pengujian	Hasil yang Diharapkan	Status	Bukti Ringkas / Catatan
T5-01	Instansiasi Sortable secara langsung	Compiler Menolak (Error) ▾	PASS ▾	Error: Sortable is abstract; cannot be instantiated

ID Skenario	Skenario Pengujian	Hasil yang Diharapkan	Status	Bukti Ringkas / Catatan
T5-02	Class Employee tanpa mengimplementasikan compare()	Compiler Menolak (Error)	PASS	Error: Employee is not abstract and does not override...
T5-03	Menguji perbandingan gaji compare() pada Employee & Manager	Mengembalikan -1, 0, atau 1...	PASS	Output -1, -1, 1, -1 sesuai nominal gaji
T5-04	Menguji kenaikan gaji raiseSalary() khusus Manager	Gaji bertambah dengan tam...	PASS	Persentase bonus otomatis ditambahkan berdasarkan currentYear - hireYear
T5-05	Instansiasi Taxable secara langsung	Compiler Menolak (Error)	PASS	Error: Taxable is abstract; cannot be instantiated
T5-06	Class Toy / Book tanpa implementasi calculateTax()	Compiler Menolak (Error)	PASS	Error: Toy is not abstract and does not override...
T5-07	Perhitungan pajak calculateTax() pada Toy dan Book	Pajak terhitung sesuai price...	PASS	Toy tax: 7200.0, Book tax: 5400.0
T5-08	Mengakses Taxable via polimorfisme interface	Method calculateTax() dapa...	PASS	Variabel Taxable taxableToy = toy berhasil mengeksekusi method

4. PERBANDINGAN ABSTRACT CLASS DAN INTERFACE (STEP 17)

Pertanyaan	Sortable (abstract class)	Taxable (interface)
Dapat dibuat object langsung?	Tidak bisa	Tidak bisa
Menggunakan extends atau implements?	extends	implements
Dapat mempunyai implementasi method konkret?	Ya, dapat memiliki method konkret dan abstract	Secara umum berisi abstract method (kecuali default/static method di versi Java modern)

Pertanyaan	Sortable (abstract class)	Taxable (interface)
Peran utamanya pada kasus praktikum?	Membentuk hubungan hierarki keluarga ("is-a") untuk pengurutan/pembandingan objek	Menyediakan kontrak kapabilitas/sifat ("can-do") yang dapat dipasang pada berbagai kelas yang berbeda
Apakah class dapat mempunyai parent class lain sekaligus?	Tidak, Java hanya mendukung single inheritance untuk class	Ya, satu class dapat mengimplementasikan banyak interface sekaligus

## 5. CHECKPOINT DAN REFLEKSI (BAGIAN 6.3)

### 1. Mengapa Sortable berguna walaupun object Sortable tidak pernah dibuat?

Sortable berguna sebagai kerangka dasar (*abstraction*) dan tipe umum (*polymorphism*). Ini memastikan bahwa semua kelas turunannya memiliki kemampuan untuk dibandingkan (`compare()`) sehingga algoritma sorting dapat mengurutkan objek apa pun yang bertipe Sortable.

### 2. Apa yang terjadi ketika Employee extends Sortable tetapi belum membuat compare()?

Program akan mengalami compile-time error. Compiler mewajibkan Employee untuk meng-override metode `compare()` atau menandai kelas Employee itu sendiri sebagai abstract.

### 3. Mengapa Manager tetap dapat menggunakan compare() tanpa menulis extends Sortable?

Karena Manager merupakan turunan dari Employee (Manager extends Employee), dan Employee sudah mewarisi Sortable. Melalui sifat pewarisan (inheritance chain), Manager secara otomatis menjadi sub tipe dari Sortable.

### 4. Mengapa class Manager extends Employee extends Sortable tidak diperlukan dan tidak valid di Java?

Java tidak mendukung multiple inheritance untuk kelas untuk menghindari ambiguitas (Diamond Problem). Selain itu, karena Employee sudah extends Sortable, Manager otomatis sudah memiliki sifat Sortable tanpa perlu ekstensi ganda.

### 5. Apa yang dijanjikan Toy ketika menulis implements Taxable?

Toy berjanji (memenuhi contract) untuk menyediakan implementasi konkrit dari seluruh method yang ada di dalam interface Taxable, yaitu method `calculateTax()`.

## 6. Mengapa Goods lebih tepat menjadi superclass bagi Food/Toy/Book, sedangkan Taxable lebih tepat menjadi interface pada model ini?

Goods mencerminkan identitas dasar atau wujud utama (is-a relationship), di mana Food, Toy, dan Book semuanya adalah Barang. Sedangkan Taxable mencerminkan kemampuan/sifat tambahan (can-do / behavior), karena tidak semua barang dikenakan pajak (contohnya Food tidak berpajak, sedangkan Toy dan Book berpajak).

## 7. Apa bukti dari compiler bahwa interface benar-benar merupakan contract?

Bukti dari compiler adalah ketika kita mencoba menghapus atau tidak mengimplementasikan method `calculateTax()` pada kelas yang implements `Taxable`, compiler akan langsung menolak dan menghasilkan error kompilasi: `Class is not abstract and does not override abstract method calculateTax() in Taxable`.

## 8. Pernyataan Refleksi

"Abstract class membantu saya ketika **ingin membuat fondasi hirarki kelas yang saling berhubungan erat dan berbagi atribut/kode umum**, sedangkan interface membantu saya ketika **ingin menambahkan kemampuan atau aturan kontrak tertentu pada berbagai kelas tanpa terikat hirarki pewarisan tunggal**."

## 6. SELF ASSESSMENT

Pernyataan	Skor (1-5)
Saya memahami mengapa abstract class tidak dapat diinstansiasi.	5
Saya dapat membuat abstract method dan implementasinya di subclass.	5
Saya memahami inheritance chain tanpa menulis extends lebih dari satu class.	5
Saya dapat membuat interface dan class yang implements interface tersebut.	5
Saya memahami bahwa implements berarti memenuhi contract method.	5
Saya dapat membedakan kapan konsep lebih cocok menjadi abstract class dan kapan menjadi interface.	5